Day-4 HW

- **Find All Permutations of a String (using swap + backtracking)—Available in GFG**
- **LeetCode 47 — Permutations II (Unique Permutations using swap approach)**

Problem

Find all permutations of a given string.

Example:

```
Input: "ABC"
Output:
ABC
ACB
BAC
BCA
CAB
CBA
```

We will learn **two standard approaches**:

1. **Using visited/map array (`mp`)**
2. **Using swap approach**

Both use **recursion + backtracking**.

☑ Approach 1 — Using `mp` (Visited Array)

Idea

- Build permutation step by step.
- Use `mp` array to track used characters.
- Store temporary string in `ds`.
- Store answers in `res`.
- Undo choice after recursion.

Steps

1. Loop over characters.
2. Pick unused character.
3. Add to `ds`.
4. Mark used.
5. Recurse.
6. Backtrack.

☑ C++ Code

```
#include <bits/stdc++.h>
using namespace std;

void solve(string &s, vector<string> &res,
           string &ds, vector<int> &mp) {

    if (ds.size() == s.size()) {
        res.push_back(ds);
        return;
    }

    for (int i = 0; i < s.size(); i++) {
        if (!mp[i]) {
            ds.push_back(s[i]);
            mp[i] = 1;

            solve(s, res, ds, mp);

            mp[i] = 0;
            ds.pop_back();
        }
    }
}

int main() {
    string s = "ABC";
    vector<string> res;
    string ds = "";
    vector<int> mp(s.size(), 0);

    solve(s, res, ds, mp);

    for (auto &x : res)
        cout << x << endl;
}
```

☑ Java Code

```
import java.util.*;

class Main {

    static void solve(String s, List<String> res,
                      StringBuilder ds, boolean[] mp) {
```

```
        if (ds.length() == s.length()) {
            res.add(ds.toString());
            return;
        }

        for (int i = 0; i < s.length(); i++) {
            if (!mp[i]) {
                ds.append(s.charAt(i));
                mp[i] = true;

                solve(s, res, ds, mp);

                mp[i] = false;
                ds.deleteCharAt(ds.length() - 1);
            }
        }
    }

    public static void main(String[] args) {
        String s = "ABC";
        List<String> res = new ArrayList<>();
        boolean[] mp = new boolean[s.length()];

        solve(s, res, new StringBuilder(), mp);

        System.out.println(res);
    }
}
```

☑ Python Code

```
def solve(s, res, ds, mp):
    if len(ds) == len(s):
        res.append(ds)
        return

    for i in range(len(s)):
        if not mp[i]:
            mp[i] = True
            solve(s, res, ds + s[i], mp)
            mp[i] = False

s = "ABC"
res = []
mp = [False] * len(s)

solve(s, res, "", mp)
print(res)
```

☑ Approach 2 — Swap Backtracking

Idea

- Fix one position.
- Swap each character with current index.
- Recurse for next index.
- Swap back.

No `mp` array needed.

Steps

```
swap
recurse
swap back
```

☑ C++ Code

```cpp
#include <bits/stdc++.h>
using namespace std;

void solve(int idx, string &ds, vector<string> &res) {
    if (idx == ds.size()) {
        res.push_back(ds);
        return;
    }

    for (int i = idx; i < ds.size(); i++) {
        swap(ds[idx], ds[i]);

        solve(idx + 1, ds, res);

        swap(ds[idx], ds[i]); // backtrack
    }
}

int main() {
    string s = "ABC";
    vector<string> res;

    solve(0, s, res);

    for (auto &x : res)
        cout << x << endl;
}
```

☑ Java Code

```java
import java.util.*;

class Main {

    static void solve(int idx, char[] ds,
```

```
                        List<String> res) {

        if (idx == ds.length) {
            res.add(new String(ds));
            return;
        }

        for (int i = idx; i < ds.length; i++) {
            char t = ds[idx];
            ds[idx] = ds[i];
            ds[i] = t;

            solve(idx + 1, ds, res);

            t = ds[idx];
            ds[idx] = ds[i];
            ds[i] = t;
        }
    }

    public static void main(String[] args) {
        String s = "ABC";
        List<String> res = new ArrayList<>();

        solve(0, s.toCharArray(), res);
        System.out.println(res);
    }
}
```

☑ Python Code

```
def solve(idx, ds, res):
    if idx == len(ds):
        res.append("".join(ds))
        return

    for i in range(idx, len(ds)):
        ds[idx], ds[i] = ds[i], ds[idx]

        solve(idx + 1, ds, res)

        ds[idx], ds[i] = ds[i], ds[idx]

s = list("ABC")
res = []
solve(0, s, res)

print(res)
```

☑ Approach Comparison

Method	Extra Space	Speed	Interview Preference
mp array	Extra array	Slight slower	Easy to understand

Method	Extra Space	Speed	Interview Preference
Swap	In-place	Faster	Interview favorite

☑ Interview Tip

If interviewer asks:
“Better approach?”

Answer:

Swap approach is better because it uses in-place modification and saves extra space.

☑ LeetCode 47 — Permutations II (Swap + Backtracking)

Goal: Return all **unique permutations** even if the array contains duplicates.

Approach (Swap + Backtracking):

1. Fix index `idx`.
2. Swap each element with current index.
3. Recurse for next index.
4. Swap back (backtrack).
5. Use a **set per level** to avoid duplicate swaps.

☑ C++ Solution

```
class Solution {
public:
    void solve(int idx, vector<int>& ds,
               vector<vector<int>>& res) {

        if (idx == ds.size()) {
            res.push_back(ds);
            return;
        }

        unordered_set<int> used;

        for (int i = idx; i < ds.size(); i++) {
            if (used.count(ds[i])) continue;
```

```
            used.insert(ds[i]);

            swap(ds[idx], ds[i]);

            solve(idx + 1, ds, res);

            swap(ds[idx], ds[i]); // backtrack
        }
    }

    vector<vector<int>> permuteUnique(vector<int>& nums) {
        vector<vector<int>> res;
        solve(0, nums, res);
        return res;
    }
};
```

☑ Java Solution

```
class Solution {

    void solve(int idx, int[] ds,
               List<List<Integer>> res) {

        if (idx == ds.length) {
            List<Integer> temp = new ArrayList<>();
            for (int x : ds) temp.add(x);
            res.add(temp);
            return;
        }

        HashSet<Integer> used = new HashSet<>();

        for (int i = idx; i < ds.length; i++) {
            if (used.contains(ds[i])) continue;
            used.add(ds[i]);

            int t = ds[idx];
            ds[idx] = ds[i];
            ds[i] = t;

            solve(idx + 1, ds, res);

            t = ds[idx];
            ds[idx] = ds[i];
            ds[i] = t;
        }
    }

    public List<List<Integer>> permuteUnique(int[] nums) {
        List<List<Integer>> res = new ArrayList<>();
        solve(0, nums, res);
        return res;
    }
}
```

☑ Python Solution

```
class Solution:
    def permuteUnique(self, nums):
        res = []

        def solve(idx):
            if idx == len(nums):
                res.append(nums[:])
                return

            used = set()

            for i in range(idx, len(nums)):
                if nums[i] in used:
                    continue
                used.add(nums[i])

                nums[idx], nums[i] = nums[i], nums[idx]

                solve(idx + 1)

                nums[idx], nums[i] = nums[i], nums[idx]

        solve(0)
        return res
```